FULL STACK DEVELOPMENT WITH MERN

iMovies

Movie Ticket Booking Website

Project Documentation

Technology Stack: React (Vite), Node.js, Express.js, MongoDB Atlas

Project Repository: https://github.com/Ananya-Jain30/iMovies

Live Website: https://imovies-website.onrender.com

1. Introduction

Project Title: iMovies – Movie Ticket Booking Website

iMovies is a web-based movie discovery and ticket-booking project developed as part of Full Stack Development with MERN. It allows users to explore movies, view showtimes, select seats, enter booking details, and receive a booking confirmation. The project demonstrates integration of a React frontend with a Node.js and Express backend and a MongoDB database.

Team Members: Ananya Jain (project developer).

2. Project Overview

2.1 Purpose

The purpose of iMovies is to provide a simple, attractive, and convenient online interface for exploring films and booking cinema seats. It demonstrates REST APIs, database operations, authentication, frontend-backend communication, and deployment.

2.2 Main Features

- Movie catalogue with posters, descriptions, genres, languages, duration, release year, ratings, and prices where available.
- Search and genre filters to help users find movies.
- Booking flow with showtime selection, seat selection, customer details, and booking confirmation.
- User registration and login using email and password.
- Password hashing with bcryptjs and JSON Web Tokens (JWT) for authentication responses.
- MongoDB storage for users, movies, shows, and bookings.
- Responsive cinema-inspired interface with animations and interactive feedback.
- Public deployment of the frontend and backend using Render.

3. Architecture

3.1 Frontend

The frontend uses React and Vite. React components and state manage the movie catalogue, booking modal, login/registration interface, seat selection, and notifications. The frontend sends HTTP requests to the backend using the API base URL configured through VITE_API_URL.

3.2 Backend

The backend uses Node.js and Express.js to expose REST API endpoints. Middleware handles JSON request bodies and cross-origin requests. Route modules separate functionality for movies, authentication, shows, and bookings. Mongoose defines schemas and communicates with MongoDB.

3.3 Database

MongoDB Atlas stores persistent application data. Mongoose models represent users, movies, shows, and bookings. A show references a movie, while a booking records customer details, selected movie/show, seats, total amount, and status. Check the model files for the exact field definitions.

Model / Collection	Purpose
User	Stores registered user details, email, and hashed password.
Movie	Stores movie information such as title, genre, language, duration, poster, and price.
Show	Stores showtime, theatre, format, date, price, movie reference, and booked seats.
Booking	Stores booking reference, customer details, selected seats, total amount, and status.

4. Setup Instructions

4.1 Prerequisites

- Node.js and npm.
- MongoDB Atlas account or another MongoDB instance.
- Git for version control.
- Visual Studio Code or another code editor.
- A modern web browser.

4.2 Clone the repository

```
git clone https://github.com/Ananya-Jain30/iMovies.git
cd iMovies
```

4.3 Install dependencies

Install dependencies for the frontend and backend in separate terminal commands:

```
cd client
npm install

cd ../server
npm install
```

4.4 Environment variables

Create a server/.env file with the required values. Use your own connection string and secret values; never commit real credentials to GitHub.

```
MONGO_URI=<your MongoDB connection string>
JWT_SECRET=<a strong secret value>
```

```
CLIENT_URL=http://localhost:5173
PORT=5000
```

For local frontend development, configure client/.env if needed:

```
VITE_API_URL=http://localhost:5000/api
```

For deployment, configure these values in Render. The frontend API URL must point to the deployed backend API, and CLIENT_URL must match the deployed frontend origin.

5. Folder Structure

```
iMovies/
├── client/
│   ├── src/
│   │   ├── App.jsx          # Main React interface and flows
│   │   ├── data.js          # Demo/fallback movie and show data
│   │   └── ...              # Styles, assets, and other frontend files
│   ├── package.json
│   └── .env.example
├── server/
│   ├── src/
│   │   ├── models/           # Mongoose models
│   │   └── routes/           # API route modules
│   ├── server.js             # Express application entry point
│   ├── seed.js               # Inserts/updates movies and showtimes
│   ├── package.json
│   └── .env.example
└── .gitignore
```

This is a high-level overview; additional files may exist in the current repository.

6. Running the Application

Run the backend and frontend in two separate terminal windows.

6.1 Backend

```
cd server
npm install
npm run dev
```

The backend normally runs at http://localhost:5000.

6.2 Frontend

```
cd client
npm install
npm run dev
```

Vite normally serves the frontend at http://localhost:5173.

6.3 Seed movie and show data

The seed script creates or updates movie records and adds showtimes for the current date. Run it only after confirming that server/.env points to the intended database.

```
cd server
npm run seed
```

7. API Documentation

The backend exposes REST endpoints under the /api prefix. The table below summarises the main routes used by the application; verify exact request fields and responses against the current route files.

Method	Endpoint	Purpose
GET	/api/health	Checks whether the backend is responding.
GET	/api/movies	Returns the movie catalogue.
GET	/api/shows?movieId=<id>	Returns shows associated with a movie.
POST	/api/auth/register	Registers a user using name, email, and password.
POST	/api/auth/login	Authenticates a user using email and password.
POST	/api/bookings	Creates a booking using customer, show, seat, and amount details.

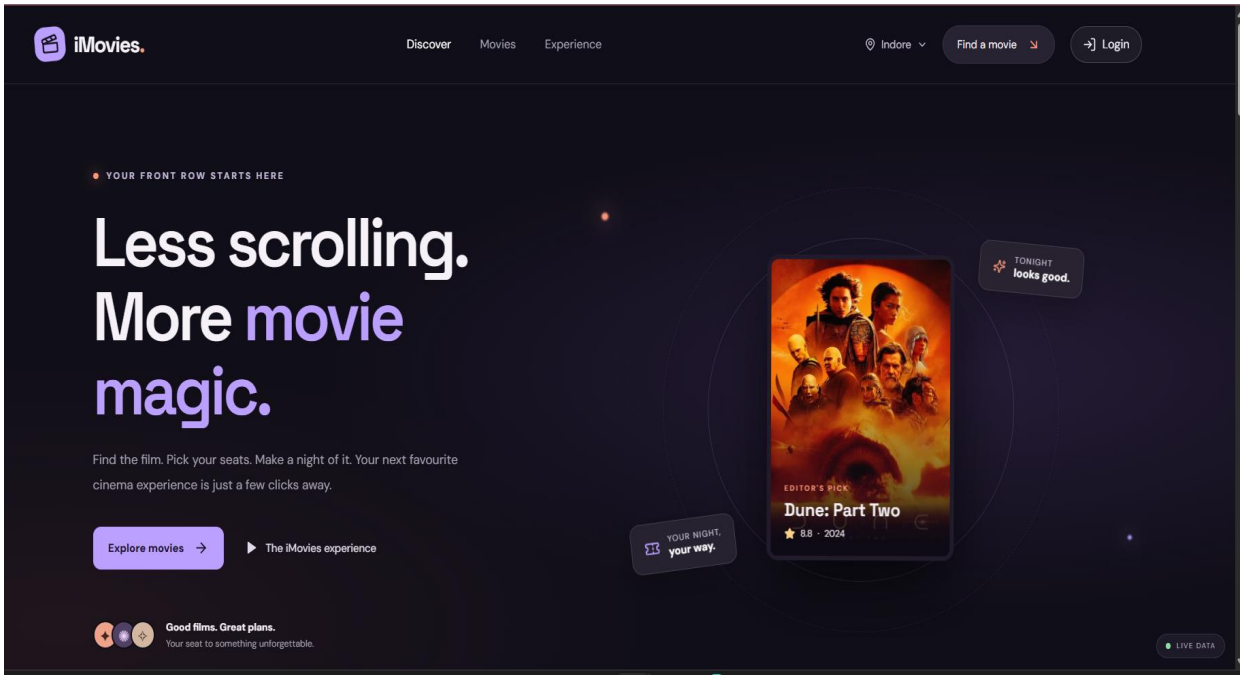
8. Authentication

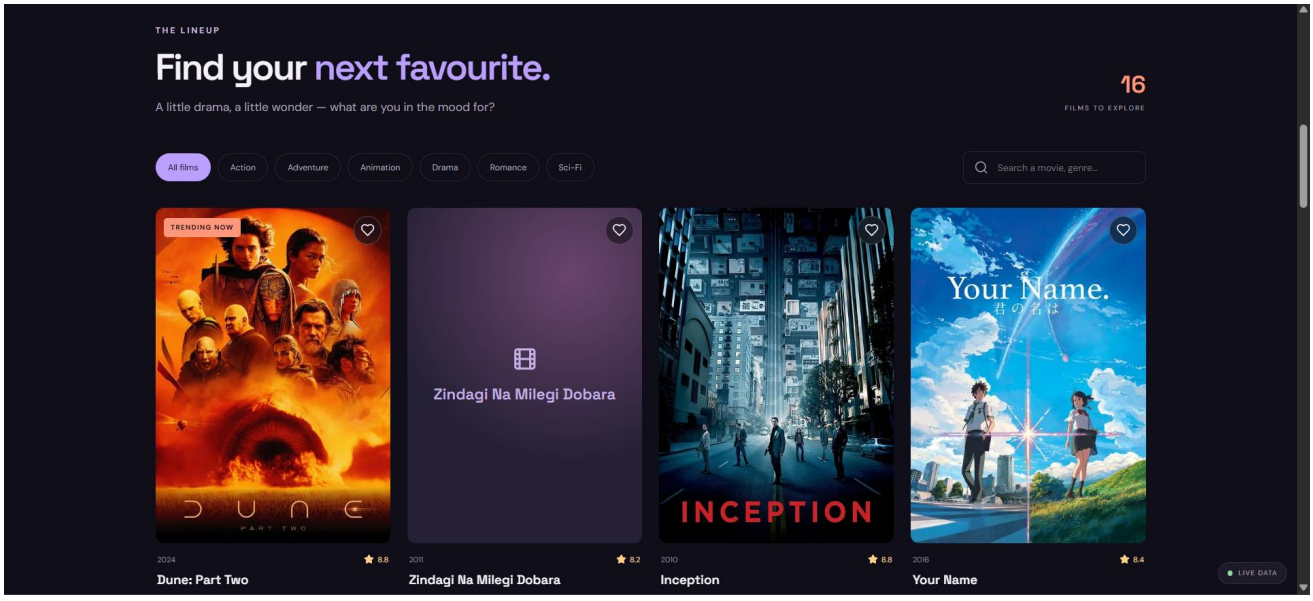
Registration and login are implemented in the authentication routes. During registration, bcryptjs hashes the password before the user is saved. During login, the submitted password is compared with the stored hash. The backend generates a JWT with a two-hour expiry. The frontend stores the returned token and user details in localStorage and uses the saved user state to display the logged-in user and logout control.

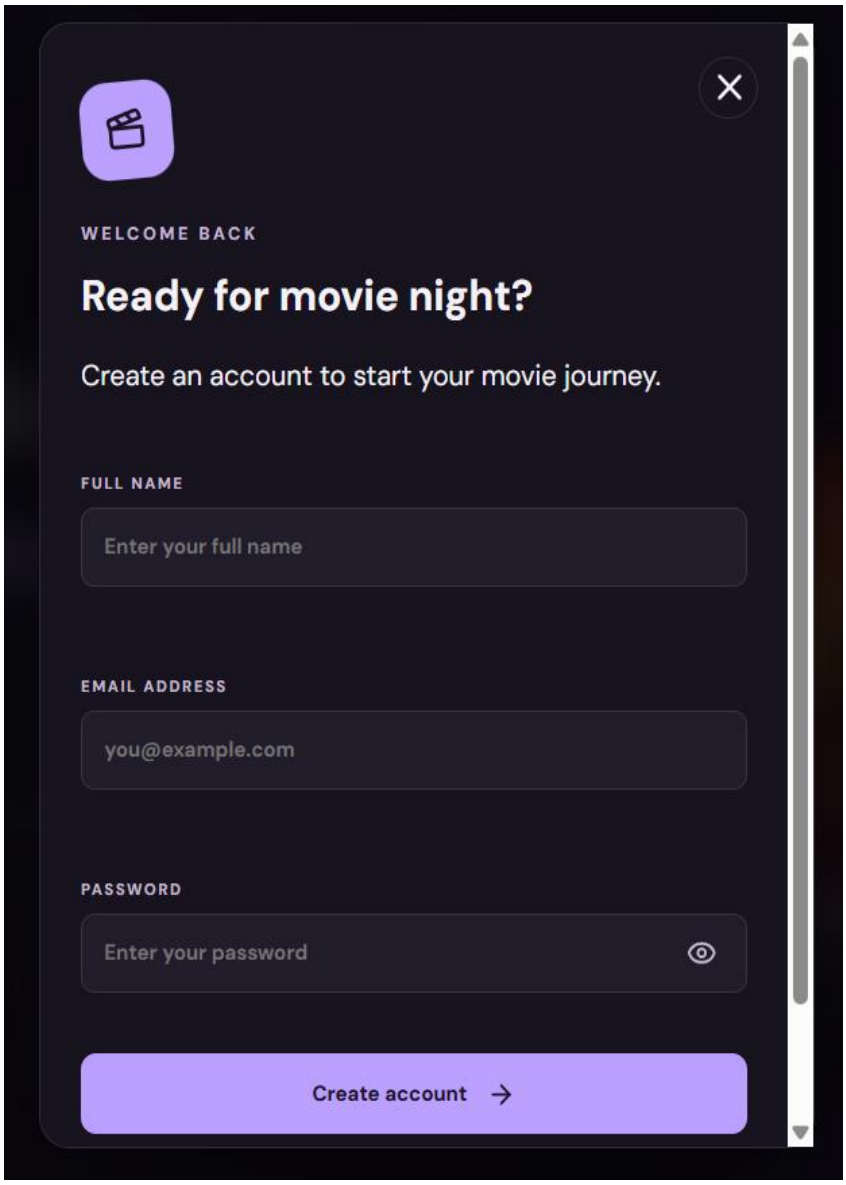
Any protected backend operation should independently verify the token on the server. Client-side state alone should not be treated as authorization.

9. User Interface

The interface uses a cinema-inspired style with a featured hero section, movie cards, genre filters, search, seat selection, booking details, and modal dialogs. The login interface supports switching between login and account registration. Visual feedback includes selected seats, validation errors, notifications, and booking confirmation.







10. Testing

Manual testing was used during development and deployment. Mark each check as completed only after verifying it on the final deployed version.

- Frontend loads successfully in a browser.
- Backend health endpoint returns a successful response.
- Movie catalogue is retrieved from the backend.
- Registration validates input and creates an account.
- Login accepts valid credentials and rejects invalid credentials.
- Showtimes are returned for the selected movie.
- Seat selection and booking confirmation complete successfully.

- Deployment environment variables point to the correct frontend and backend URLs.

Tools used include a browser, CMD/terminal, Visual Studio Code, Render dashboard, and direct API endpoint checks. No automated test suite is described unless one is added to the repository.

11. Screenshots or Demo

https://youtu.be/H_evtxp-Z-8

12. Known Issues

- The application depends on the availability of the deployed backend and MongoDB database.
- Render free services may take time to wake after inactivity.
- Showtimes are generated for the current date by the seed script; production use would require a more complete schedule-management workflow.
- The booking experience is a demonstration and does not collect payment.
- The frontend includes demo/fallback data and may show a preview message if live show data cannot be retrieved.

13. Future Enhancements

- Add a user dashboard with previous booking history.
- Add booking cancellation and ticket retrieval.
- Integrate a secure payment gateway in a production-ready version.
- Improve seat availability synchronization to prevent duplicate seat bookings.
- Add city selection, theatre filters, and date-based showtime browsing.
- Add password reset and email verification

Conclusion

iMovies demonstrates the development and deployment of a full-stack movie ticket booking website using the MERN technology stack. It combines a React user interface, an Express REST API, MongoDB persistence, user authentication, and public deployment, and provides a foundation for further booking and account features.